实验五 软件设计模式

一、实验目的

1. 列举工厂方法模式、适配器模式、代理模式、模板方法模式和迭代器模式的特点。
2. 分析具体问题，选择合适的软件设计模式进行设计，解释不同模式的适用场景。
3. 根据实验指导写出每个模式的案例代码并进行调试运行。
4. 使用 EA 画出每个设计模式的 UML 图。

二、实验内容

1. 工厂方法模式的案例代码与 UML 图；
2. 适配器模式的案例代码与 UML 图；
3. 代理模式的案例代码与 UML 图；
4. 模板方法模式的案例代码与 UML 图；
5. 迭代器模式的案例代码与 UML 图；

三、实验步骤

工厂模式在一些设计模式的书中分为简单工厂模式、工厂方法模式和抽象工厂模式三类，也有把工厂方法模式划分到抽象工厂模式的，认为工厂方法是抽象工厂模式的特例的一种，就是只有一个要实现的产品接口。

3.1 工厂方法模式简单实现

1. 应用背景

小成有一间仅生产 A 类产品塑料加工厂，随着客户需求的变化，客户需要生产 B 类产品；改变原有塑料加工厂的配置和变化非常困难，假设下一次客户需要再发生变化，再次改变将增大非常大的成本。

2. 解决方案

小成决定置办**塑料分厂** B 来生产 B 类产品。

步骤 1：创建抽象工厂类，定义具体工厂的公共接口

```
abstract class Factory{
    public abstract Product Manufacture();
}
```

步骤 2：创建抽象产品类 ，定义具体产品的公共接口；

山东建筑大学计算机科学与技术学院

```
abstract class Product{
    public abstract void Show();
}
```

步骤 3：创建具体产品类（继承抽象产品类）， 定义生产的具体产品；

```
//具体产品 A 类
class ProductA extends   Product{
  @Override
  public void Show() {
    System.out.println("生产出了产品 A");
  }
}

//具体产品 B 类
class ProductB extends   Product{
  @Override
  public void Show() {
    System.out.println("生产出了产品 B");
  }
}
```

步骤 4：创建具体工厂类（继承抽象工厂类），定义创建对应具体产品实例的方法；

```
//工厂 A 类 - 生产 A 类产品
class FactoryA extends Factory{
  @Override
  public Product Manufacture() {
    return new ProductA();
  }
}

//工厂 B 类 - 生产 B 类产品
class FactoryB extends Factory{
  @Override
  public Product Manufacture() {
    return new ProductB();
  }
}
```

步骤 5：外界通过调用具体工厂类的方法，从而创建不同具体产品类的实例

山东建筑大学 计算机科学与技术学院

```
//生产工作流程
public class FactoryPattern {
    public static void main(String[] args){
        //客户要产品 A
        FactoryA mFactoryA = new FactoryA();
        mFactoryA.Manufacture().Show();

        //客户要产品 B
        FactoryB mFactoryB = new FactoryB();
        mFactoryB.Manufacture().Show();
    }
}
```

输出结果

生产出了产品 A

生产出了产品 C

3.2 适配器模式简单实现

1. 类适配器模式

Target 角色给出需要的目标接口，而 Adaptee 类则是需要被转换的对象，继承并实现接口

```
//目标接口
interface Target
{
    public void request();
}
//适配者接口
class Adaptee
{
    public void specificRequest()
    {
        System.out.println("适配者中的业务代码被调用！");
    }
}
//类适配器类
class ClassAdapter extends Adaptee implements Target
{
    public void request()
    {
        specificRequest();
```


山东建筑大学计算机科学与技术学院

```
        }
    }
    //客户端代码
    public class ClassAdapterTest
    {
        public static void main(String[] args)
        {
            System.out.println("类适配器模式测试：");
            Target target = new ClassAdapter();
            target.request();
        }
    }
```

2. 对象适配器模式

与类适配器相同的一点就是都是把被适配的类的 API 转换成为目标类的 API，不同的是对象的适配器模式不是使用继承关系连接到 Adaptee 类，而是使用代理关系到 Adaptee 类

```
    //目标接口
    interface Target
    {
        public void request();
    }

    //适配者接口
    class Adaptee
    {
        public void specificRequest()
        {
            System.out.println("适配者中的业务代码被调用！");
        }
    }

    //对象适配器类
    class ObjectAdapter implements Target
    {
        private Adaptee adaptee;
        public ObjectAdapter(Adaptee adaptee)
        {
            this.adaptee=adaptee;
        }
        public void request()
        {
            adaptee.specificRequest();
```


山东建筑大学 计算机科学与技术学院

```
        }
    }
    //客户端代码
    public class ObjectAdapterTest
    {
        public static void main(String[] args)
        {
            System.out.println("对象适配器模式测试：");
            Adaptee adaptee = new Adaptee();
            Target target = new ObjectAdapter(adaptee);
            target.request();
        }
    }
```

3.3 代理模式简单实现

代理模式也称为委托模式、结构化模式，就好像现实生活中请律师代理诉讼一样，本质上是为其他对象提供一个代理以控制对这个对象的访问。

联想到请律师，通过当事人的辩护、证据以及请求方法，都可以用律师这个代理类来实现，简单来说就是律师充当了这个代理的角色。

代理分为动态代理以及静态代理，静态代理是代码运行前代理类的 class 编译文件就已经存在了，而动态代理则是通过反射机制动态生成代理者对象，在 code 阶段不知道代理谁，java 提供了一个便捷的动态代理接口 InvocationHandler，实现接口重写其调用方法 invoke

```
    //抽象主题
    interface Subject
    {
        void Request();
    }
    //真实主题
    class RealSubject implements Subject
    {
        public void Request()
        {
            System.out.println("访问真实主题方法...");
        }
    }
    //代理
    class Proxy implements Subject
    {
        private RealSubject realSubject;
        public void Request()
```


山东建筑大学 计算机科学与技术学院

```
        {
            if (realSubject==null)
            {
                realSubject=new RealSubject();
            }
            preRequest();
            realSubject.Request();
            postRequest();
        }
        public void preRequest()
        {
            System.out.println("访问真实主题之前的预处理。");
        }
        public void postRequest()
        {
            System.out.println("访问真实主题之后的后续处理。");
        }
    }
    // 客户端
    public class ProxyTest
    {
        public static void main(String[] args)
        {
            Proxy proxy=new Proxy();
            proxy.Request();
        }
    }
```

其他模式都可以见到代理模式的影子，大多数结构化模式都是对代理模式的一种针对性优化。

JDK 动态代理与 CGLib 动态代理均是实现 Spring AOP 的基础。在 Spring 的 AOP 编程中，如果加入容器的目标对象有实现接口，用 JDK 代理；如果目标对象没有实现接口，用 CGlib 代理。

3.4 模板模式简单实现

模板方法模式需要注意抽象类与具体子类之间的协作。它用到了虚函数的多态性技术以及“不用调用我，让我来调用你”的反向控制技术。

出国留学手续一般经过以下流程：索取学校资料，提出入学申请，办理因私出国护照、出境卡和公证，申请签证，体检、订机票、准备行装，抵达目标学校等，其中有些业务对各个学校是一样的，但有些业务因学校不同而不同，所以比较适合用模板方法模式来实现。

山东建筑大学计算机科学与技术学院

在本实例中，先定义一个出国留学的抽象类 StudyAbroad，里面包含了一个模板方法 TemplateMethod()，该方法中包含了办理出国留学手续流程中的各个基本方法，其中有些方法的处理由于各国都一样，所以在抽象类中就可以实现，但有些方法的处理各国是不同的，必须在其具体子类（如美国留学类 StudyInAmerica）中实现。如果再增加一个国家，只要增加一个子类就可以了。

```
//抽象类: 出国留学
abstract class StudyAbroad
{
    public void TemplateMethod() //模板方法
    {
        LookingForSchool(); //索取学校资料
        ApplyForEnrol();      //入学申请
        ApplyForPassport(); //办理因私出国护照、出境卡和公证
        ApplyForVisa();       //申请签证
        ReadyGoAbroad();      //体检、订机票、准备行装
        Arriving();           //抵达
    }
    public void ApplyForPassport()
    {
        System.out.println("三.办理因私出国护照、出境卡和公证：");
        System.out.println("1）持录取通知书、本人户口簿或身份证向户口所在地公安机关 \
申请办理因私出国护照和出境卡。");
        System.out.println("2）办理出生公证书，学历、学位和成绩公证，经历证书，\
亲属关系公证，经济担保公证。");
    }
    public void ApplyForVisa()
    {
        System.out.println("四.申请签证：");
        System.out.println("   1）准备申请国外境签证所需的各种资料，包括个人学历、成绩单、\
            工作经历的证明；个人及家庭收入、资金和财产证明；家庭成员的关系证明等；");
        System.out.println("   2）向拟留学国家驻华使(领)馆申请入境签证。\
            申请时需按要求填写有关表格，递交必需的证明材料，缴纳签证。\
            有的国家(比如美国、英国、加拿大等)在申请签证时会要求申请人\
            前往使(领)馆进行面试。");
    }
    public void ReadyGoAbroad()
    {
        System.out.println("五.体检、订机票、准备行装：");
        System.out.println("1）进行身体检查、免疫检查和接种传染病疫苗；");
        System.out.println("2）确定机票时间、航班和转机地点。");
    }
```


山东建筑大学 计算机科学与技术学院

```
    public abstract void LookingForSchool();//索取学校资料
    public abstract void ApplyForEnrol();     //入学申请
    public abstract void Arriving();          //抵达
}
//具体子类: 美国留学
class StudyInAmerica extends StudyAbroad
{
    @Override
    public void LookingForSchool()
    {
        System.out.println("一.索取学校以下资料：");
        System.out.println("  1）对留学意向国家的政治、经济、文化背景和教育体制、\
            学术水平进行较为全面的了解；");
        System.out.println("  2）全面了解和掌握国外学校的情况，包括历史、学费、学制、\
            专业、师资配备、教学设施、学术地位、学生人数等；");
        System.out.println("  3）了解该学校的住宿、交通、医疗保险情况如何；");
        System.out.println("  4）该学校在中国是否有授权代理招生的留学中介公司？");
        System.out.println("  5）掌握留学签证情况；");
        System.out.println("  6）该国政府是否允许留学生合法打工？");
        System.out.println("  8）毕业之后可否移民？");
        System.out.println("  9）文凭是否受到我国认可？");
    }
    @Override
    public void ApplyForEnrol()
    {
        System.out.println("二.入学申请：");
        System.out.println("  1）填写报名表；");
        System.out.println("  2）将报名表、个人学历证明、最近的学习成绩单、推荐信、\
            个人简历、托福或雅思语言考试成绩单等资料寄往所申请的学校；");
        System.out.println("  3）为了给签证办理留有充裕的时间，建议越早申请越好，\
            一般提前 1 年就比较从容。");
    }
    @Override
    public void Arriving()
    {
        System.out.println("六.抵达目标学校：");
        System.out.println("  1）安排住宿；");
        System.out.println("  2）了解校园及周边环境。");
    }
}
//客户端代码
public class StudyAbroadTest
{
    public static void main(String[] args)
```


山东建筑大学计算机科学与技术学院

```
        {
            StudyAbroad tm=new StudyInAmerica();
            tm.TemplateMethod();
        }
    }
```

3.5 迭代器模式简单实现

迭代器模式是通过将聚合对象的遍历行为分离出来，抽象成迭代器类来实现的，其目的是在不暴露聚合对象的内部结构的情况下，让外部代码透明地访问聚合的内部数据。

```
//抽象迭代器
interface Iterator
{
    Object first();
    Object next();
    boolean hasNext();
}
//具体迭代器
class ConcreteIterator implements Iterator
{
    private List<Object> list=null;
    private int index=-1;
    public ConcreteIterator(List<Object> list)
    {
        this.list=list;
    }
    public boolean hasNext()
    {
        if(index<list.size()-1)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
    public Object first()
    {
        index=0;
        Object obj=list.get(index);;
        return obj;
    }
    public Object next()
```

山东建筑大学 计算机科学与技术学院

```
        {
            Object obj=null;
            if(this.hasNext())
            {
                obj=list.get(++index);
            }
            return obj;
        }
    }
    //抽象聚合
    interface Aggregate
    {
        public void add(Object obj);
        public void remove(Object obj);
        public Iterator getIterator();
    }
    //具体聚合
    class ConcreteAggregate implements Aggregate
    {
        private List<Object> list=new ArrayList<Object>();
        public void add(Object obj)
        {
            list.add(obj);
        }
        public void remove(Object obj)
        {
            list.remove(obj);
        }
        public Iterator getIterator()
        {
            return(new ConcreteIterator(list));
        }
    }

    // 客户端代码
    public class IteratorPatternTest
    {
        public static void main(String[] args)
        {
            Aggregate ag=new ConcreteAggregate();
            ag.add("山东大学");
            ag.add("山东建筑大学");
            ag.add("济宁学院");
```

山东建筑大学 计算机科学与技术学院

```
        System.out.print("聚合的内容有：");
        Iterator it=ag.getIterator();
        while(it.hasNext())
        {
            Object ob=it.next();
            System.out.print(ob.toString()+"\t");
        }
        Object ob=it.first();
        System.out.println("\nFirst："+ob.toString());
    }
}
```

四、实验要求

本次实验，参照步实验步骤骤三，完成以下内容：

1. 搭建开发环境 Eclipse，输入上述示例代码并进行一定的修改完善，使代码能够顺利运行，将运行结果截屏复制到实验报告；

2. 将调试后的代码复制到实验报告，注意板式；

3. 使用 EA 绘制相应模式的 UML 图；

4. 根据实验步骤逐项完成任务，并按撰写规范完成实验报告。报告不得抄袭或拷贝，一旦被发现以零分计。

五、实验总结

1. 此次实验完成情况

2. 遇到的问题及其解决办法

3. 存在问题

4. 下一步的努力方向

请将实验报告以“班级-学号-姓名-实验5”格式命名并按时提交到指定系统。

山东建筑大学 计算机科学与技术学院